Attention all you need(https://arxiv.org/pdf/1706.03762)

Revolution papers

1. **Sequence-to-sequence learning with a neural network**

   Where big sentences lose their previous context

2. **Neural machine translation by jointly learning to align and translate**

   This paper introduces an attention mechanism to keep the previous context in the sentence

   Transfer learning is not applicable(limitation).

3. **Then comes the Transformer paper**

Timeline

2000 - 2014 —> LSTM/RNN

2014 —-> attention

2017 —-> Transformer

2018 —-> BERT/GPT

2018-2020 —-> Vision Transformer

2021 —-> Gen Ai

2022 —-> ChatGPT

COMPREHENSIVE SURVEY ON TRANSFORMER

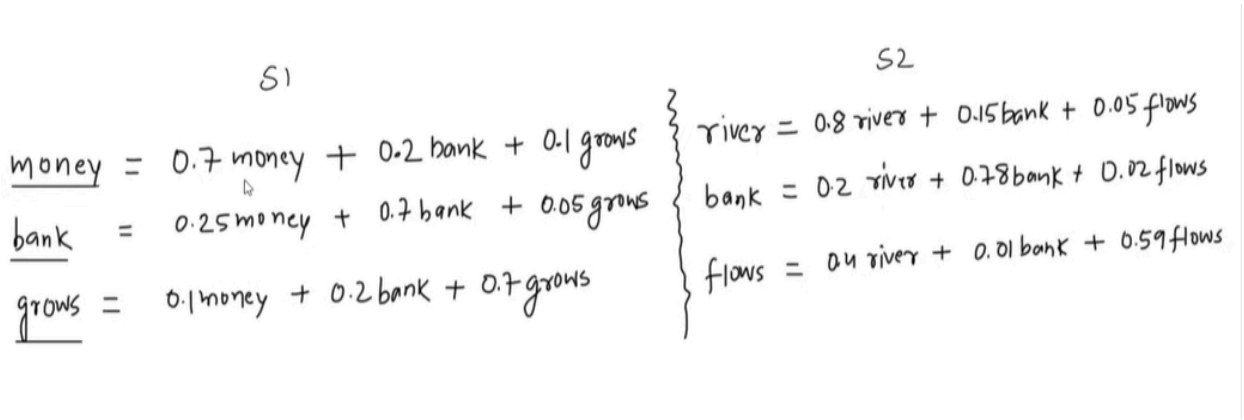
Paper link : (https://arxiv.org/pdf/2306.07303)

Dive into the paper

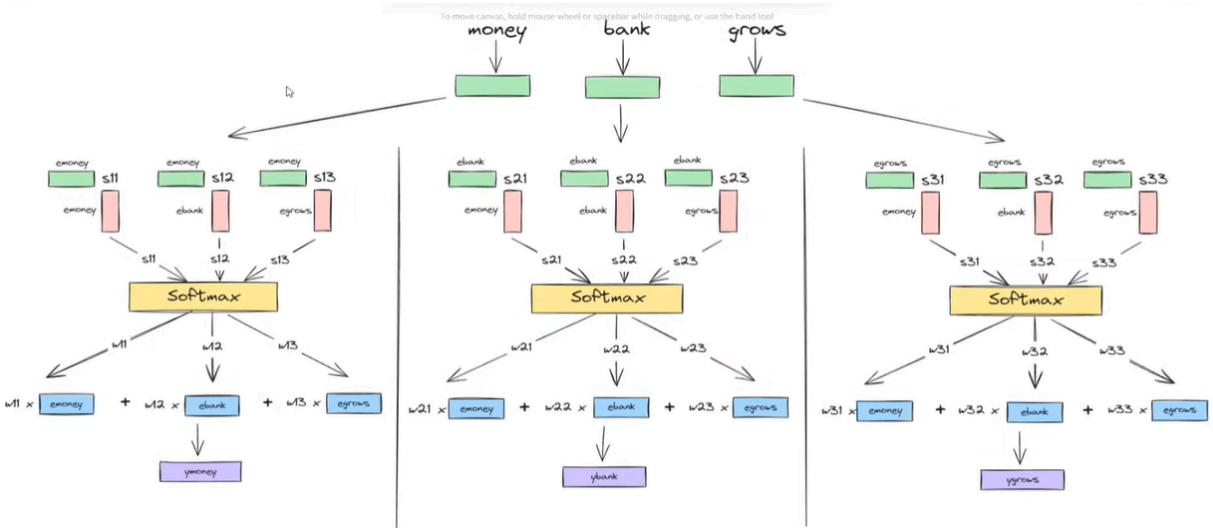
Importance of self-attention:

The embedding technique is static; it can be tilted to some particular value, apple as a fruit or Apple is a company[0.8, 0.2] in context. If in a whole paragraph apples arise frequently as a fruit, then the company

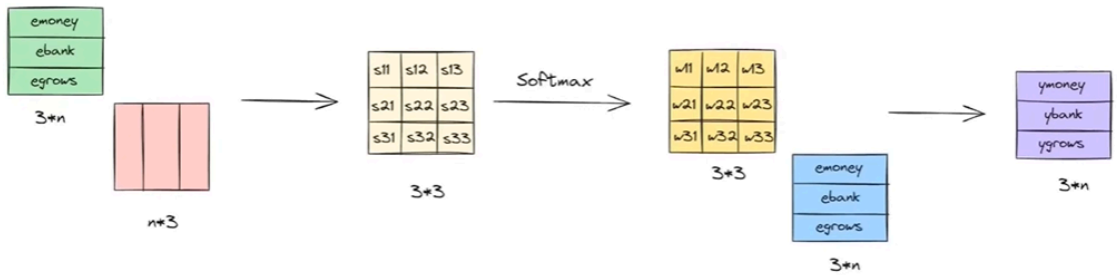
Self Attention:
Below, each word represents the word embedding. This is the main funda of self-attention. Instead, only use the embedding vector of the word it giving priority to each word with semantic meaning.
The numbers 0.7, 0.2, and 0.1 for the first word come from the vector dot of (Emoney dot Emoney^T), (Emoney dot Ebank^T), and (Emoney dot Egrows^T). And the same for other words in the sentence.



Below is the full context of self-attention

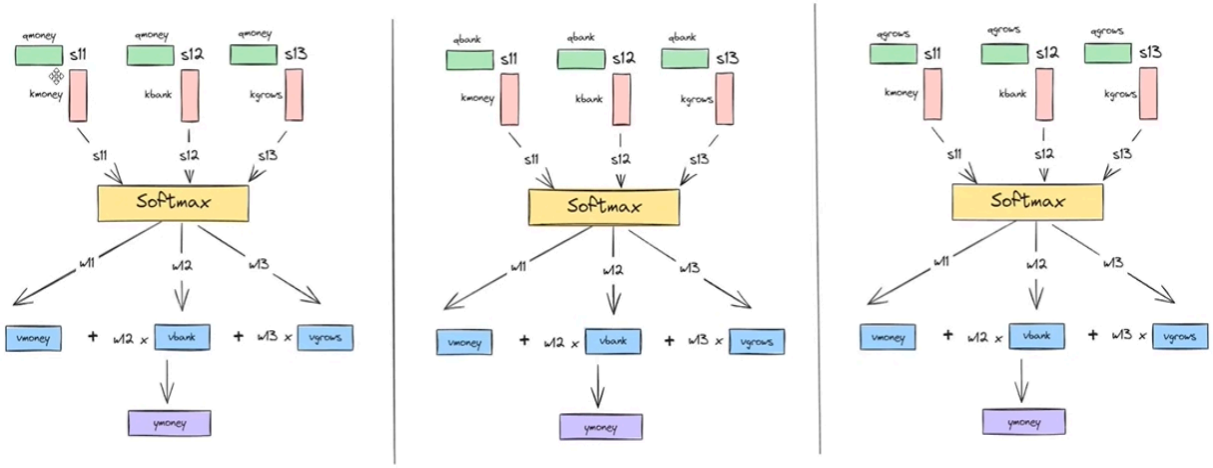


A transformer can take all sequences in parallel



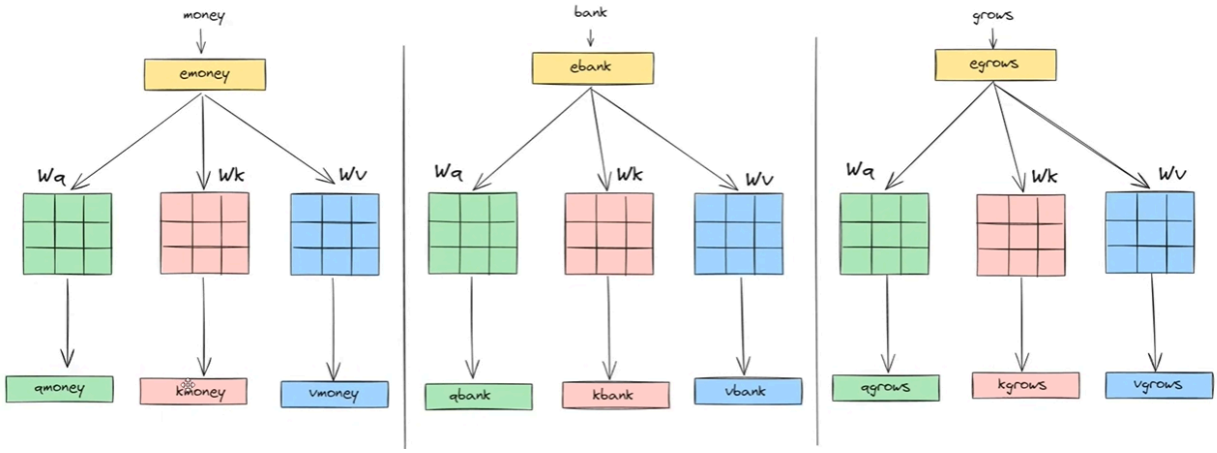
In this process, there are no learning parameters
This is a simple self-attention technique that can perform only general context, not task-specific context. Let's put some learnable parameters so that this technique does task-specific work.
Each embedding plays three roles: query, key, and value, like a dictionary.

Instead of only using the embedding vector of each word, first find these three values of each word and keep doing the above operation.



Now the question is, how can we find those three values?

Ans: linear transform



Wa, Wk, Wv are all learnable; they will learn from data.

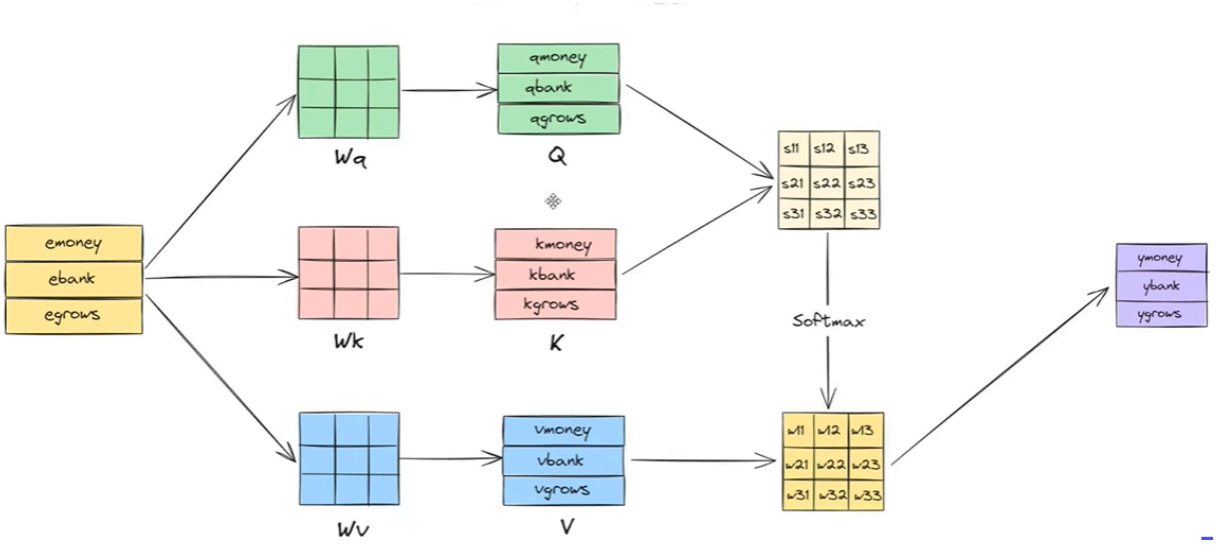
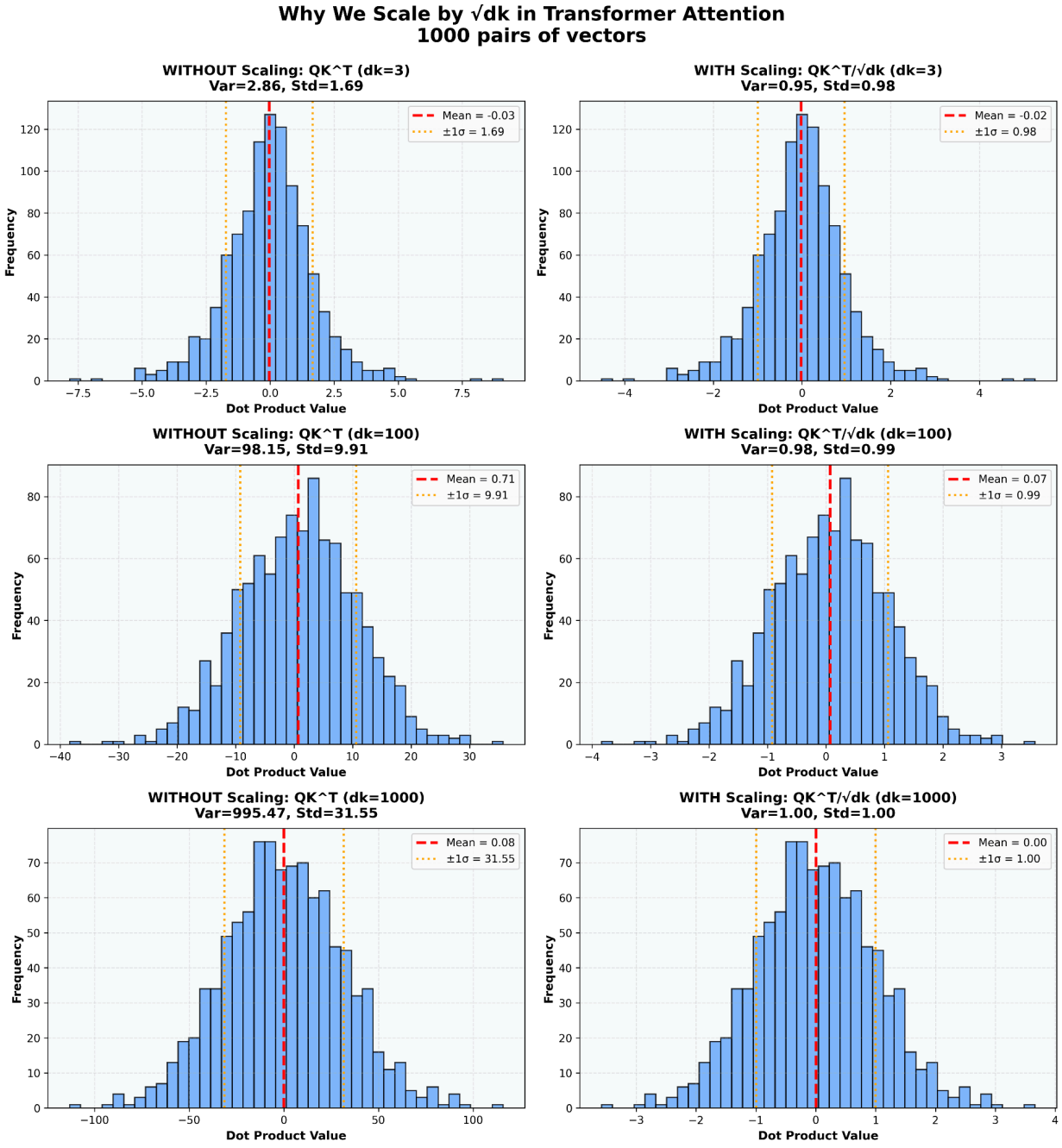


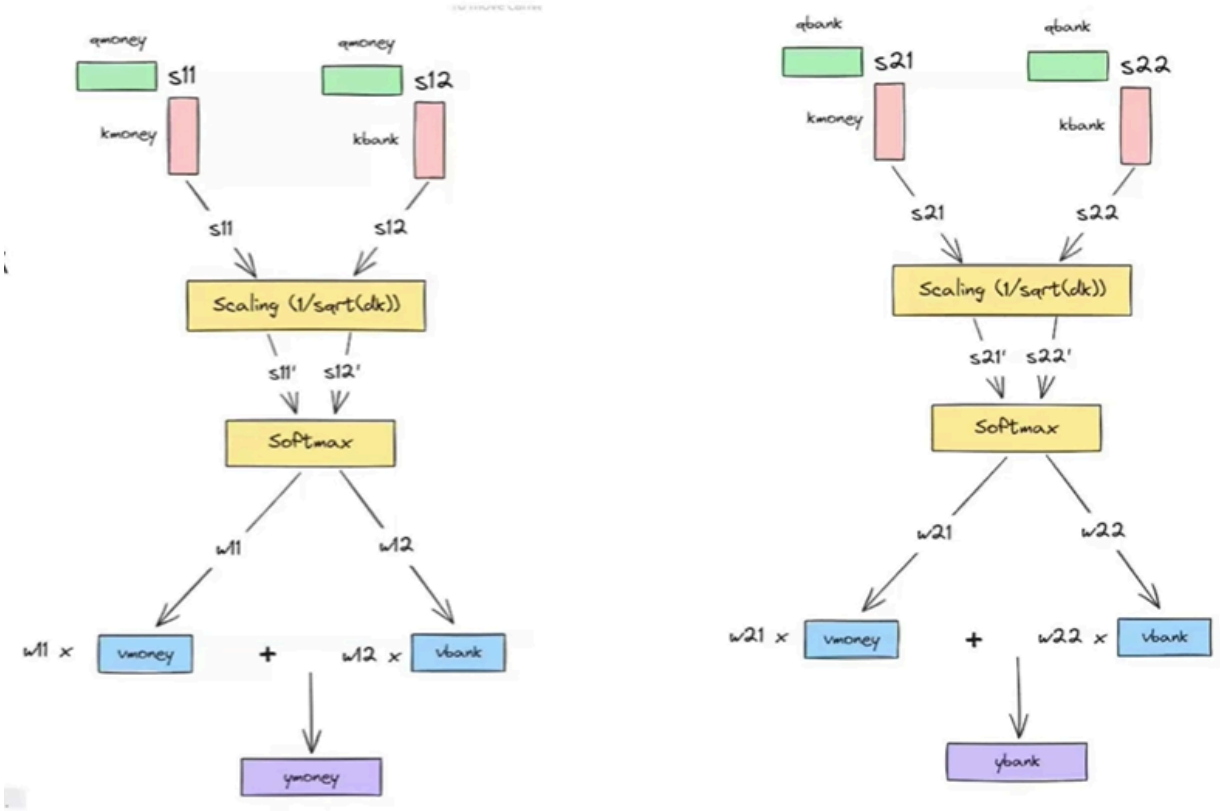
Fig: Full attention

Scaled Dot-Product Attention:

Attention(Q, K, V) = softmax((Q × K^T) / √d□) × V

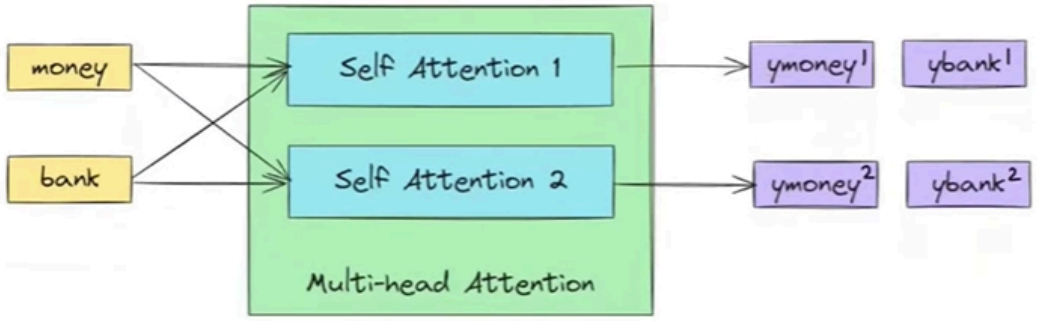
Why do we need to do this?



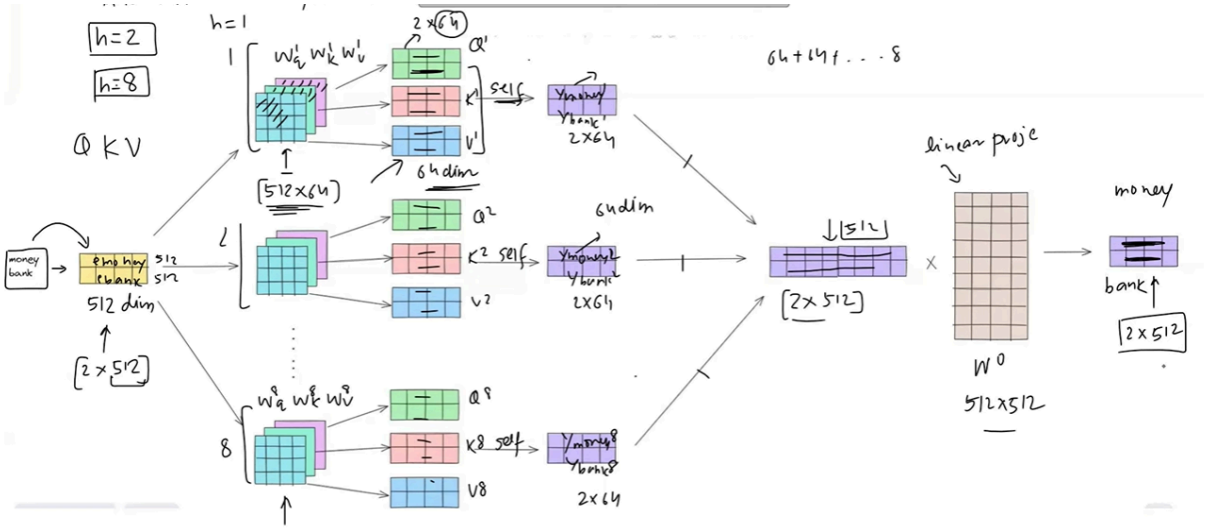


Multi-head attention:

Multi-head attention = multiple viewpoints on the same information.



In the paper author uses 8 multi-head self-attention



According to the paper.
In step 3, dimension reduction is applied to 64 from 512 dimensions (1/8th) to reduce computational cost for a single head self-attention, while maintaining more precise context than single self-attention.

Viz tool - [bertviz_tutorial.ipynb - Colab]

🧩 Positional Encoding

What & Why

Problem: Transformers process all tokens in parallel → no sense of word order

Solution: Add positional information to token embeddings

Formula: Input = Token_Embedding + Positional_Encoding

PE(pos, 2i) = sin(pos / 10000^(2i/d_model))

PE(pos, 2i+1) = cos(pos / 10000^(2i/d_model))

🎯 Why Layer Normalization is Used

Transformers use **Layer Normalization (LayerNorm)** instead of **Batch Normalization (BatchNorm)** due to the unique characteristics of sequential data processing.

Key Reasons:

Aspect	Batch Normalization	Layer Normalization
Dependency	Requires batch statistics	Independent of batch size
Sequence Length	Affected by variable lengths	Handles variable lengths naturally
Inference	Needs running statistics	Works with single samples
Best For	CNNs with large batches	Transformers, RNNs, small batches

Why Transformers Prefer LayerNorm:

1. **Variable Sequence Lengths**: Text sequences vary greatly in length (3 words vs 500 words). LayerNorm doesn't depend on sequence length statistics.
2. **Small Batch Sizes**: During inference, you might process just one sentence. LayerNorm works perfectly with batch_size=1.
3. **Stable Training**: LayerNorm normalizes within each token's hidden features, providing consistent gradients regardless of batch composition.
4. **Independence**: Each token is normalized independently, making the model more robust to batch size and sequence length variations.

📐 Layer Normalization Formula

For an input vector representing a single token:

$x = [x_1, x_2, ..., x_a]$

where **d** is the hidden dimension (e.g., 512, 768, 1024), the Layer Normalization is computed as:

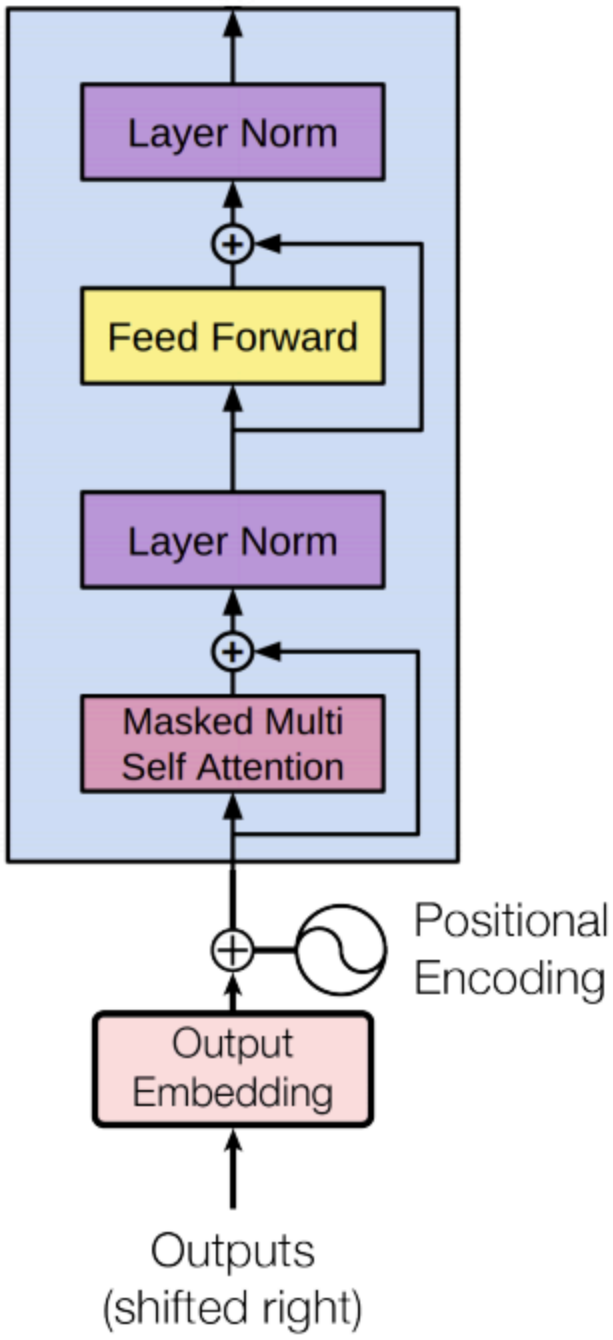
$$\text{LayerNorm}(x) = ((x - \mu) / \sigma) \cdot \gamma + \beta$$

Components:

1. Mean (μ)

$\mu = (1/d) \times \Sigma(x_i)$ for i=1 to d

Transformer Architecture
Encoder part



In the paper, they use 6 encoder parts on a stack.
Common question on this encoder

1. **Why use residual connection?**
2. **Why use FFNN?**
3. **Why use a 6 encoder?**

Summary:

Component	Purpose	Key Formula / Concept	Benefits
Residual Connection	**Stabilizes deep learning and gradient flow**	**Output = LayerNorm(x + Sublayer(x))**	**Prevents vanishing gradients; faster convergence**
Feed Forward Network (FFN)	**Applies a non-linear transformation to each token**	**FFN(x) = ReLU(xW_1 + b_1)W_2 + b_2)**	**Adds non-linearity and expands representation**
6 Encoder Layers	**Builds hierarchical representations**	**Stack of (Self-Attention + FFN) × 6**	**Captures deeper semantic and contextual features**

Decoder Part

Most of the NLP tasks are autoregressive. That means it takes reference from the previous prediction, like sentence translation.

The transformer decoder is autoregressive at inference time and non-autoregressive at training time.

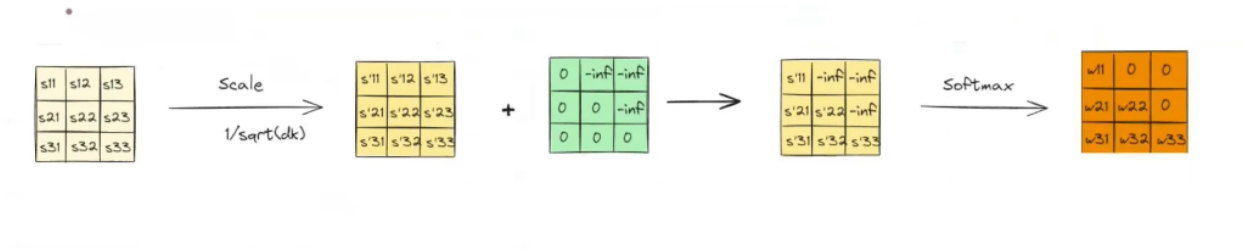
Masked Self-Attention in Transformer Decoder

Purpose

- The decoder generates output sequences step by step (e.g., words in a sentence).
- To prevent the model from "seeing" future tokens during training, a mask is applied in self-attention.

How It Works

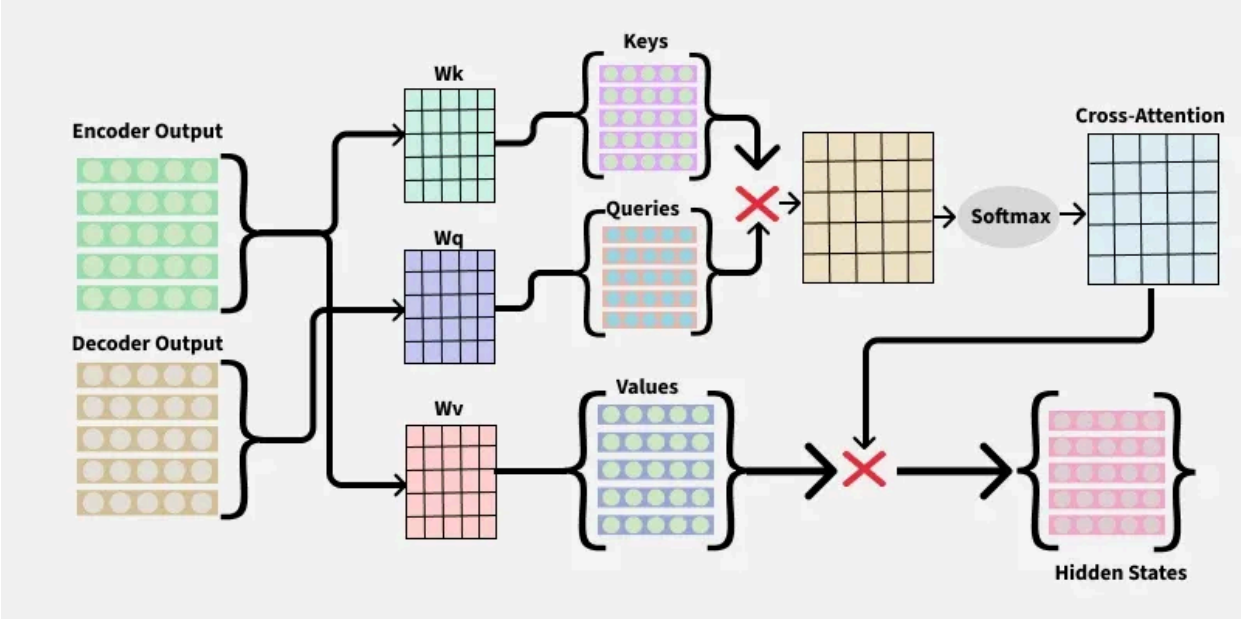
1. Each position in the decoder attends only to:
    - Itself, and
    - Previous positions in the output sequence.
2. A triangular (causal) mask



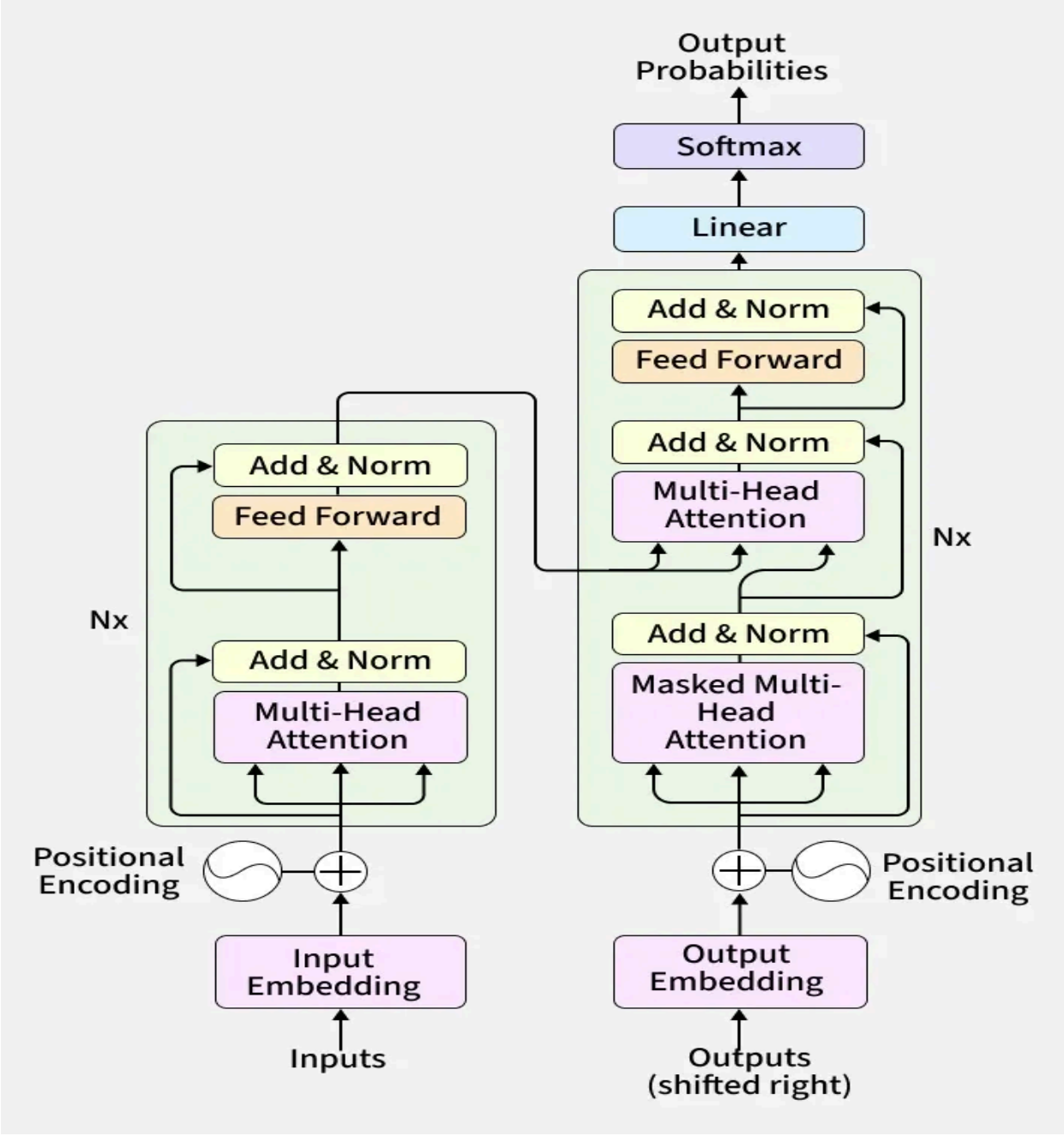
3. The softmax then ensures that attention weights for future tokens are zero.

Cross Attention:

Cross-attention enables the decoder to use information from the encoder outputs to generate context-aware predictions.



Transformer full architecture:



Summary of the decoder during training and inference

Phase	Input to Decoder	Mask Used	Generation Type	Output

Training	Shifted target sequence	Yes (causal mask)	Parallel	Predicts next token for all positions
Inference	Previously generated tokens	Yes (causal mask)	Sequential (one-by-one)	Next token at each step

Congratulations, Indro, you just finished the Transformers architecture.